

Université Hassan I
École Nationale des Sciences Appliquées Berrechid

Rapport de Travaux Pratiques

Modélisation UML et Gestion de Versions

Module : Maquette et Ingénierie Numérique

Réalisé par : Saad Benhaimer

Encadrant : Pr. Hrimech Hamid

Année universitaire : 2025/2026

Table des matières

1	Introduction	2
2	Cas 1 : Plateforme E-commerce	2
2.1	Description	2
2.2	Diagramme de cas d'utilisation	2
2.3	Diagramme de séquence	4
2.4	Diagramme d'activités	4
2.5	Gestion de versions avec Git	6
2.5.1	Initialisation du dépôt	6
2.5.2	Création des branches de fonctionnalités	6
2.5.3	Développement d'une fonctionnalité	6
2.5.4	Fusion dans la branche develop	6
3	Cas 2 : Gestion Universitaire	7
3.1	Description	7
3.2	Travail collaboratif	8
3.3	Workflow Collaboratif Git	8
3.3.1	Création des branches	8
3.3.2	Développement et Push	8
3.3.3	Pull Request	8
3.3.4	Mise à jour locale	9
4	Cas 3 : Gestion des conflits Git	10
4.1	Objectif	10
4.2	Simulation complète	10
4.2.1	Création du dépôt	10
4.2.2	Développeur 1	10
4.2.3	Développeur 2	10
4.2.4	Fusion provoquant le conflit	10
4.3	Conflit obtenu	10
4.4	Résolution du conflit	11
5	Cas 4 : Système de Réservation d'Hôtel	12
5.1	Description	12
6	Conclusion	14

1 Introduction

Ce projet présente la modélisation UML et la gestion de versions Git pour différents cas d'étude, permettant d'allier la conception logicielle à une méthodologie de développement collaborative et rigoureuse.

2 Cas 1 : Plateforme E-commerce

2.1 Description

La plateforme permet aux clients de consulter des produits, de gérer leur panier et d'effectuer des paiements de manière sécurisée.

FIGURE 1 – Architecture globale de la plateforme E-commerce

2.2 Diagramme de cas d'utilisation

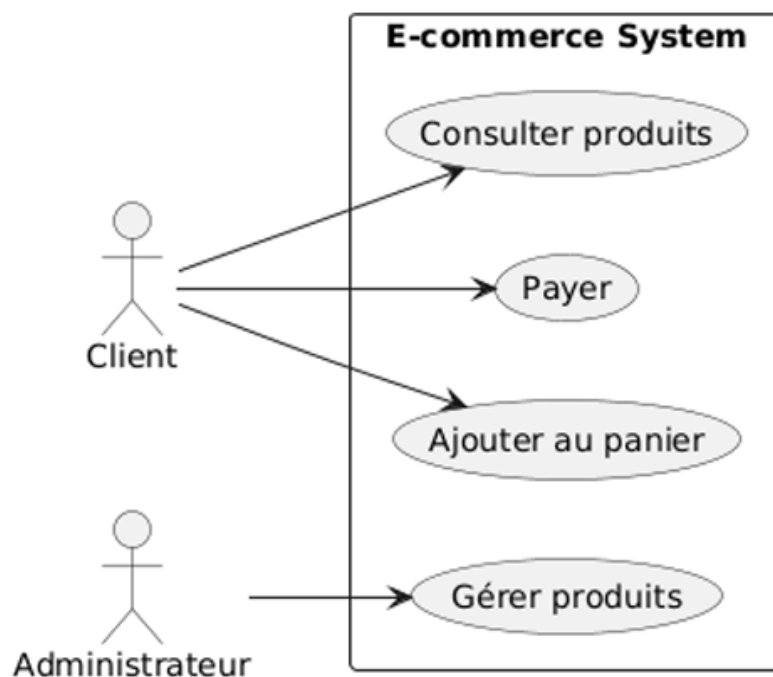


FIGURE 2 – Diagramme de cas d'utilisation

FIGURE 3 – Diagramme de cas d'utilisation - E-commerce

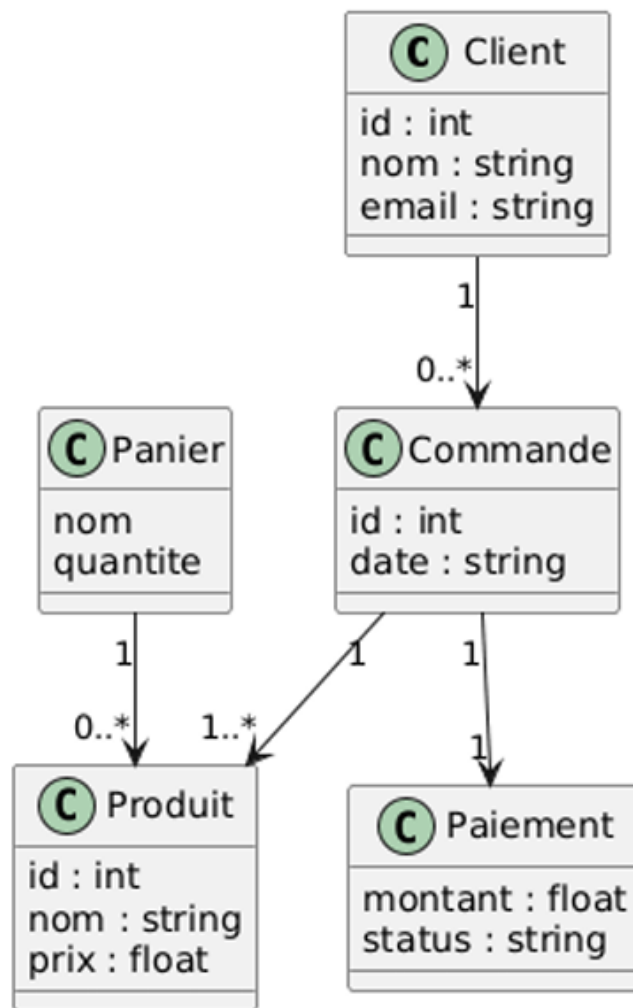


FIGURE 4 – Diagramme de classe

2.3 Diagramme de séquence

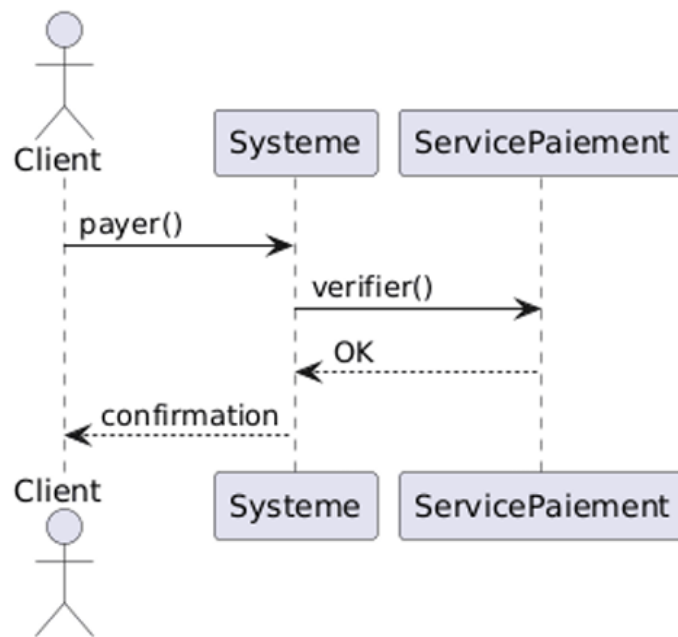


FIGURE 5 – Diagramme de sequence

2.4 Diagramme d'activités

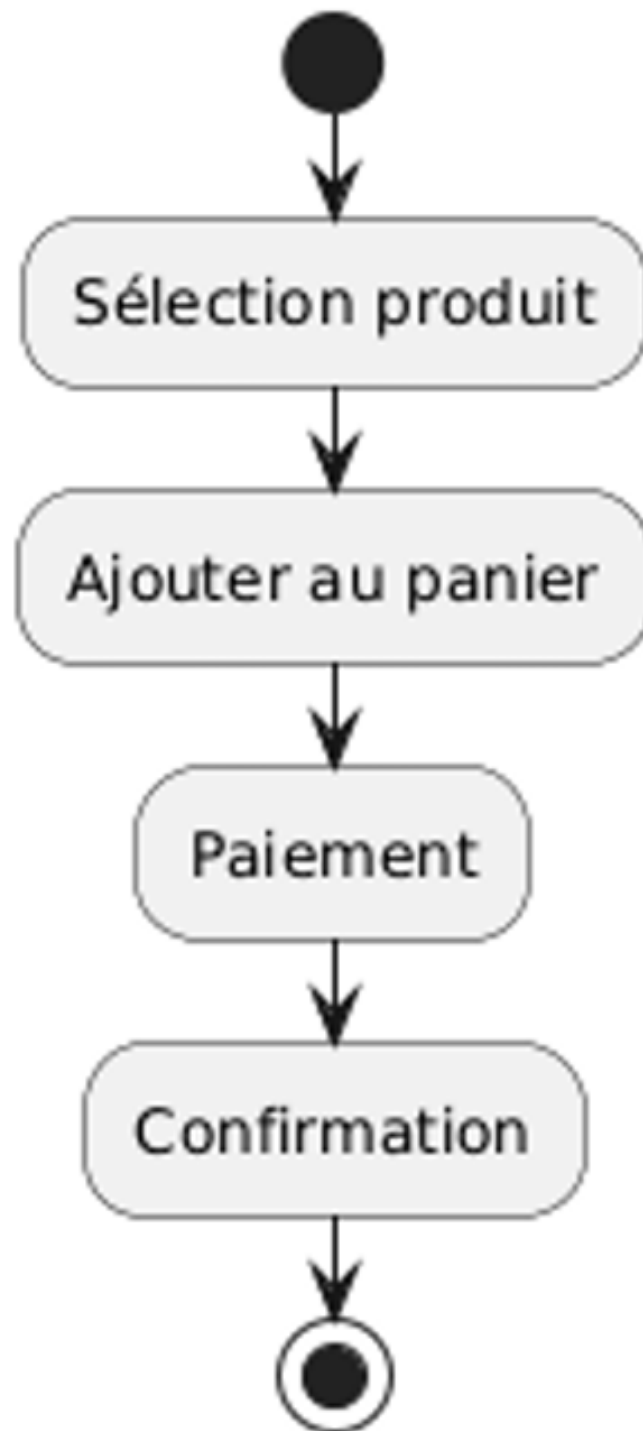


FIGURE 6 – Diagramme d'activités

2.5 Gestion de versions avec Git

Le cycle de développement des fonctionnalités de la plateforme a été orchestré à l'aide de Git, en suivant le workflow standard Git Flow (branches de fonctionnalités dérivées de **develop**).

2.5.1 Initialisation du dépôt

L'initialisation et la configuration initiale du dépôt principal s'effectuent ainsi :

```
1 git init
2 git add .
3 git commit -m "Initial commit with UML diagrams"
4 git branch -M main
5 git checkout -b develop
```

2.5.2 Création des branches de fonctionnalités

Chaque module de la plateforme fait l'objet d'une branche isolée pour garantir la stabilité de la branche de développement :

```
1 git checkout -b feature/produits
2 git checkout develop
3 git checkout -b feature/panier
4 git checkout develop
5 git checkout -b feature/paiement
6 git checkout develop
```

2.5.3 Développement d'une fonctionnalité

Exemple d'implémentation et de publication des modifications pour le module de paiement :

```
1 git checkout feature/paiement
2 echo "Fonction paiement implémentée" > paiement.txt
3 git add paiement.txt
4 git commit -m "Implémentation paiement"
5 git push origin feature/paiement
```

2.5.4 Fusion dans la branche develop

Une fois la fonctionnalité validée, elle est intégrée à la branche de développement commune :

```
1 git checkout develop
2 git merge feature/paiement
3 git push origin develop
```

3 Cas 2 : Gestion Universitaire

3.1 Description

Cette application permet d'assurer le suivi et la gestion administrative des étudiants, des enseignants ainsi que l'affectation et la planification des différents cours.

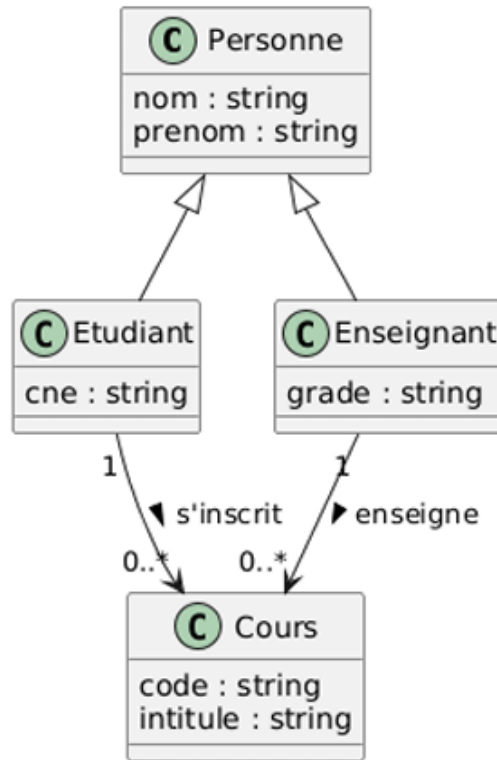


FIGURE 7 – Diagramm de Classe

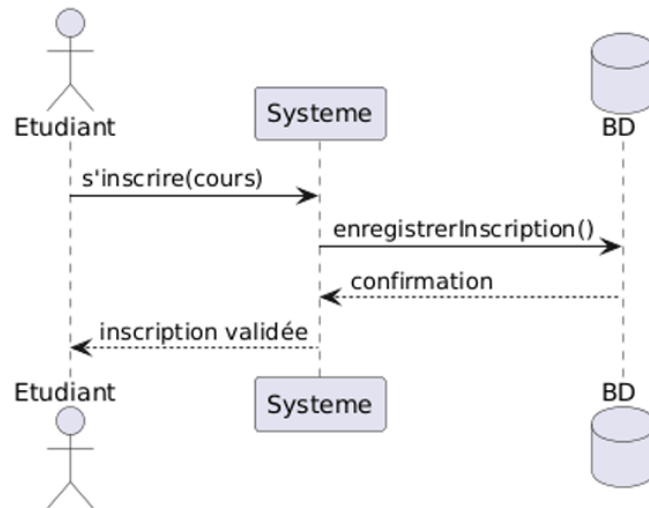


FIGURE 8 – Diagramme de séquence

3.2 Travail collaboratif

L'organisation du travail en équipe repose sur des principes stricts de gestion de flux de production :

- Création de branches **feature/*** dédiées pour chaque tâche isolée.
- Synchronisation régulière via des **push** vers le dépôt distant GitHub.
- Soumission obligatoire d'une **Pull Request** (PR) avant toute intégration.
- Validation croisée par les pairs puis fusion finale.

3.3 Workflow Collaboratif Git

3.3.1 Création des branches

Chaque membre de l'équipe initialise sa branche de travail locale à partir de la dernière version stable :

```

1 git checkout -b feature/etudiant
2 git checkout -b feature/enseignant
3 git checkout -b feature/cours
  
```

3.3.2 Développement et Push

Une fois la modification apportée localement, les fichiers sont indexés, archivés et poussés sur le serveur distant :

```

1 echo "Gestion Etudiant" > etudiant.txt
2 git add .
3 git commit -m "Ajout gestion tudiant "
4 git push origin feature/etudiant
  
```

3.3.3 Pull Request

Après le push vers GitHub :

- Création d'une **Pull Request** ciblant la branche **develop**.

- Phase d’inspection et de validation du code.
- Fusion (*Merge*) dans la branche **develop**.
- Suppression de la branche de fonctionnalité devenue obsolète.

3.3.4 Mise à jour locale

Pour entamer un nouveau cycle de développement, la base locale doit être resynchronisée :

```
1 git checkout develop
2 git pull origin develop
```

4 Cas 3 : Gestion des conflits Git

4.1 Objectif

Comprendre l'origine, le mécanisme de détection et la méthodologie de résolution manuelle des conflits lors de fusions concurrentes sous Git.

4.2 Simulation complète

4.2.1 Création du dépôt

```
1 git init
2 echo "Contenu initial" > fichier.txt
3 git add .
4 git commit -m "Initial commit"
```

4.2.2 Développeur 1

Le premier développeur modifie la ligne sur sa propre branche alternative :

```
1 git checkout -b dev1
2 echo "Version A de Dev1" > fichier.txt
3 git add fichier.txt
4 git commit -m "Modification par Dev1"
```

4.2.3 Développeur 2

Simultanément, le second développeur produit une modification divergente sur la même ligne depuis la souche initiale :

```
1 git checkout main
2 git checkout -b dev2
3 echo "Version B de Dev2" > fichier.txt
4 git add fichier.txt
5 git commit -m "Modification par Dev2"
```

4.2.4 Fusion provoquant le conflit

La collision survient immédiatement lorsque l'on tente d'unifier les deux historiques divergents sans hiérarchie claire :

```
1 git checkout dev1
2 git merge dev2
```

4.3 Conflit obtenu

Lors de la fusion de la branche `dev2` dans la branche `dev1`, Git détecte que les deux développeurs ont modifié la même ligne du fichier `fichier.txt`. Le système ne peut pas déterminer automatiquement quelle version conserver et génère alors un conflit d'intégration.

Le contenu du fichier se transforme pour inclure les balises de conflit :

```
1 <<<<<< HEAD
2 Version A de Dev1
3 =====
4 Version B de Dev2
5 >>>>>> dev2
```

Les marqueurs affichés par Git permettent d'identifier précisément la provenance des lignes en opposition :

- <<<<<< HEAD : version de la branche courante (**dev1**).
- ===== : séparateur strict entre les deux versions concurrentes.
- >>>>>> dev2 : version provenant de la branche entrante (**dev2**).

Pour résoudre le conflit, le développeur doit modifier manuellement le fichier, supprimer l'intégralité de ces marqueurs techniques et arbitrer pour la version finale souhaitée.

4.4 Résolution du conflit

Après modification manuelle du fichier `fichier.txt` pour obtenir le résultat attendu, la finalisation de la fusion s'exécute ainsi :

```
1 git add fichier.txt
2 git commit -m "R é s o l u t i o n   d u   c o n f l i t"
```

5 Cas 4 : Système de Réservation d'Hôtel

5.1 Description

Application permettant la réservation de chambres d'hôtel, incluant le suivi des disponibilités, des profils clients et de la facturation associée.

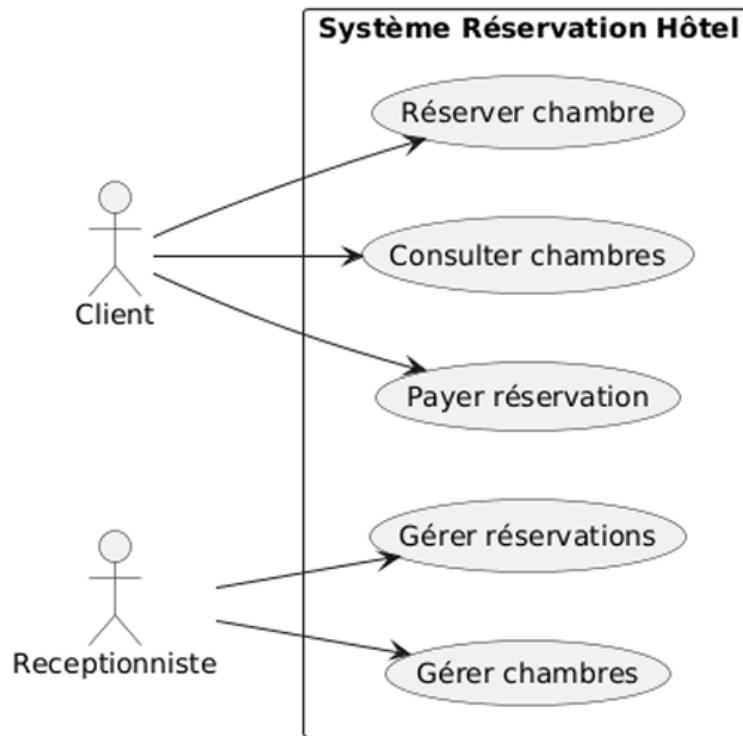


FIGURE 9 – Use case

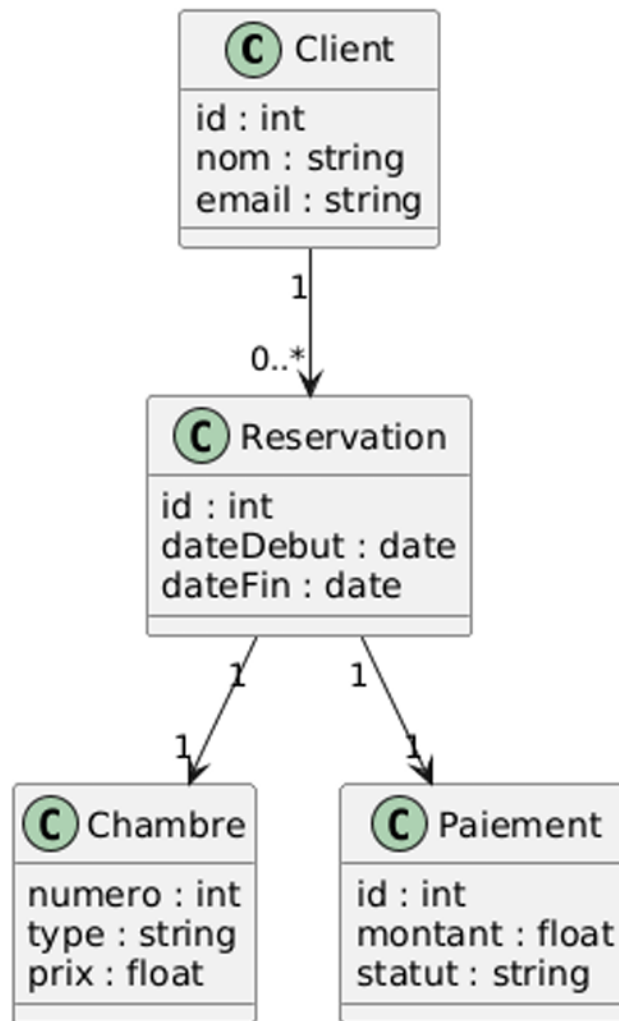


FIGURE 10 – Class

6 Conclusion

En somme, ce projet a permis de concrétiser l’alliance indispensable entre la modélisation conceptuelle avec UML et la gestion de configuration logicielle via Git et GitHub.

À travers l’analyse des différents cas d’étude, l’élaboration des diagrammes (cas d’utilisation, de classes et de séquence) a offert une vision claire et structurée du comportement architectural des systèmes. Parallèlement, l’application concrète des workflows Git (GitFlow, gestion des branches, Pull Requests) et la résolution manuelle de conflits de fusion ont fourni les clés d’une collaboration d’équipe efficace et sécurisée.

Cette approche globale nous offre une immersion concrète dans les méthodologies industrielles actuelles. Elle démontre qu’une conception logicielle rigoureuse, associée à une gestion de version maîtrisée, constitue le socle indispensable à la réussite, à la qualité et à la maintenabilité de tout projet d’ingénierie numérique.